Exp No:28

COUNTING SEMAPHORE IN SYSTEMC

AIM:

To implement and simulate a counting semaphore in SystemC to allow multiple tasks to access a limited number of resources simultaneously.

PROCEDURE:

1. Initialize the semaphore count to 2, representing two available resources.
2. Create three concurrent tasks using SC_THREAD.
3. Make each task check whether the semaphore count is greater than zero before accessing a resource.
4. Decrease the semaphore count when a task accesses a resource and increase it after the task completes its execution.
5. Run the simulation for 10 seconds and observe the resource count to verify that no more than two tasks access resources simultaneously.

PROGRAM:

```
#include <systemc.h>


SC_MODULE(counting_semaphore) {


int sem;


void task() {

while(true) {

wait(1, SC_SEC);


if(sem > 0) {

sem--;

cout << "Task Accessing Resource at "

<< sc_time_stamp()

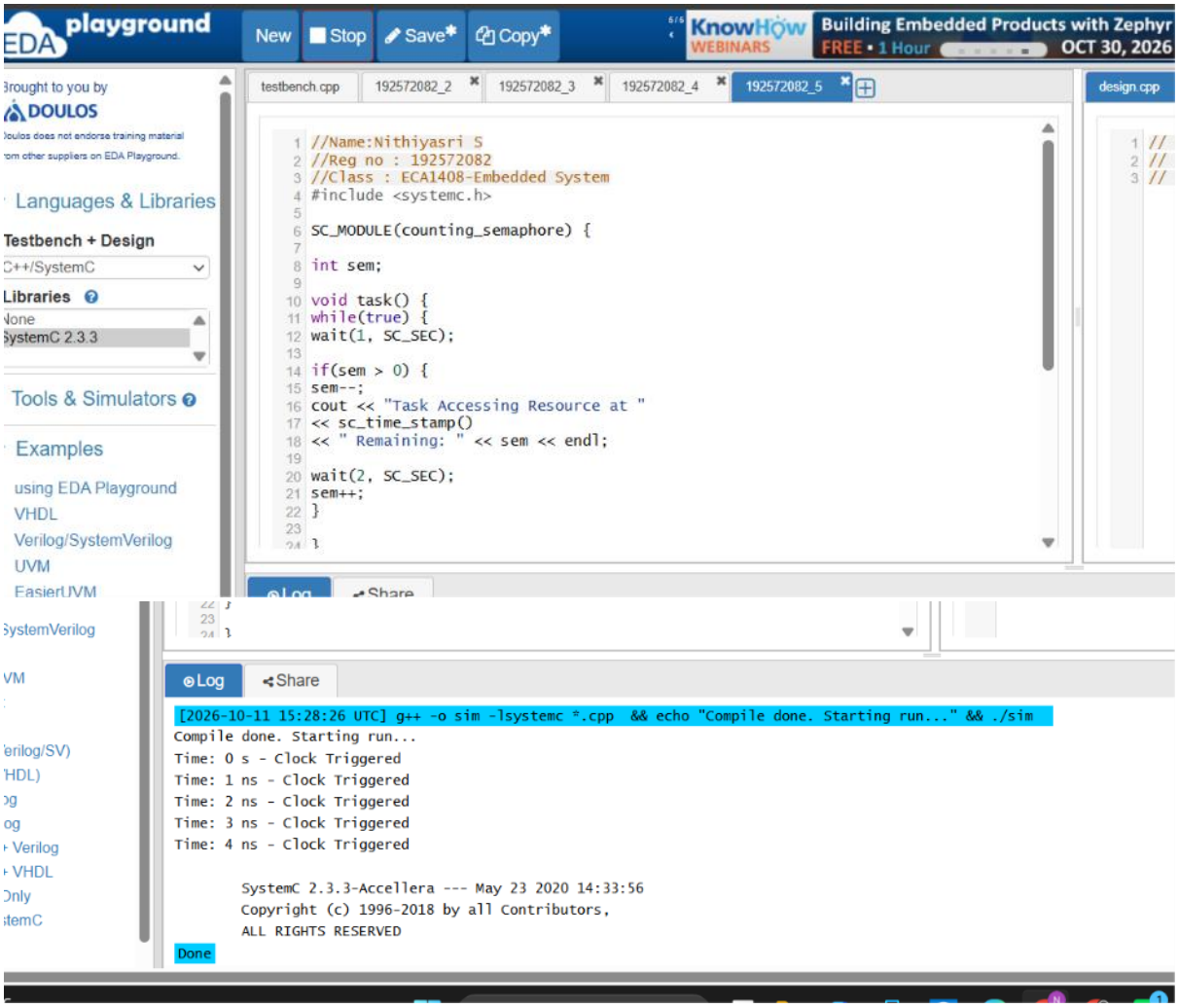
<< " Remaining: " << sem << endl;
```

```
wait(2, SC_SEC);
sem++;
}

}
}

SC_CTOR(counting_semaphore) {
sem = 2;
SC_THREAD(task);
SC_THREAD(task);
SC_THREAD(task);
}
};

int sc_main(int argc, char* argv[]) {
counting_semaphore obj("Counting");
sc_start(10, SC_SEC);
return 0;
}
```



RESULT:

Thus, the counting semaphore was implemented and simulated using SystemC, allowing a maximum of two tasks to access resources simultaneously while maintaining the resource count.